Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин
Дисциплина «Жизненный цикл разработки программного обеспечения»

ОТЧЕТ
по лабораторной работе №1
на тему

ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО «Утилита для информирования пользователя о нормальном артериальном давлении»

Студент гр. 434701 М. Ю. Кузьминчук

Преподаватель О. М. Внук

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Интегрированная среда разработки и паттерн MVC 4
1.1 Состав интегрированной среды разработки 4
1.2 Приемы работы в интегрированной среде разработки 5
1.3 Паттерн MVC. Активная и пассивная модель 5
2 Разработка приложения 7
2.1 Выбор языка программирования и среды разработки 7
2.2 Структура проекта 7
2.3 Реализация каркаса MVC с активной моделью 8
2.4 Алгоритм расчета нормального давления 9
3 Результаты работы программы 11
Заключение 15
Список использованных источников 16
Приложение А (обязательное) Листинг кода 17

ВВЕДЕНИЕ

Целью лабораторной работы является изучение методов работы с интегрированной средой разработки программного обеспечения, получение навыка разработки в выбранной среде приложения с графическим интерфейсом пользователя, а также изучение архитектурного паттерна MVC в вариантах с пассивной и активной моделью.

Интегрированная среда разработки сопровождает программиста на большинстве этапов жизненного цикла программного обеспечения: при написании исходного текста, сборке, запуске, тестировании и отладке программы, а также при сохранении изменений в системе управления версиями. Использование паттерна MVC уже на этапе проектирования позволяет разделить хранение данных, их обработку и отображение, благодаря чему программу проще сопровождать и дорабатывать.

Исходное задание по варианту 10: разработать утилиту для информирования пользователя о нормальном артериальном давлении. На главной форме поместить кнопку «Ввести данные». В отдельном окне запросить у пользователя дату рождения (не используя стандартный элемент ввода даты), вес и текущее давление. Последние введенные данные запоминать в главной форме и восстанавливать при повторном вводе. После этого на главной форме отобразить результат – «идеальное давление» для возраста и веса и оценку текущего давления человека (повышенное, пониженное, норма). Если введены некорректные данные – показать сообщение об ошибке. Добавить каркас MVC с активной моделью [1].

Дополнительно требуется выбрать объектно-ориентированный язык программирования и среду разработки, установить и настроить программное обеспечение, ознакомиться с его функциональными возможностями и средствами отладки, создать проект под выданный вариант задания и описать разработанный каркас MVC.

1 ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ И ПАТТЕРН MVC

1.1 Состав интегрированной среды разработки

Интегрированная среда разработки, IDE (англ. Integrated Development Environment) – это программный комплекс, который объединяет в одном приложении инструменты, необходимые программисту для создания программного обеспечения. Вместо того чтобы по отдельности запускать текстовый редактор, компилятор, компоновщик и отладчик, разработчик работает в едином окне, где все эти средства согласованы между собой и используют общее описание проекта.

Среди наиболее распространенных сред разработки можно выделить Microsoft Visual Studio, Visual Studio Code, IntelliJ IDEA, CLion, Eclipse и Qt Creator. Большинство из них поддерживают несколько языков программирования, а набор возможностей расширяется за счет подключаемых модулей.

Типичная интегрированная среда разработки включает следующие компоненты:

– редактор исходного кода с подсветкой синтаксиса, автодополнением, навигацией и средствами рефакторинга;

– компилятор или интерпретатор, преобразующий исходный текст в исполняемый код (для языка C++ – MSVC, GCC, Clang);

– систему автоматизации сборки, которая по описанию проекта определяет порядок компиляции и компоновки (MSBuild, CMake, qmake);

– средства запуска программы и отладчик, позволяющий останавливать выполнение и исследовать состояние программы;

– средства интеграции с системами управления версиями, прежде всего с Git;

– механизм расширений, через который подключаются дополнительные инструменты, например редакторы диаграмм UML и SysML.

Многие среды также содержат визуальный конструктор интерфейса, обозреватель классов и встроенные средства запуска модульных тестов. Состав интегрированной среды разработки изображен на рисунке 1.1.

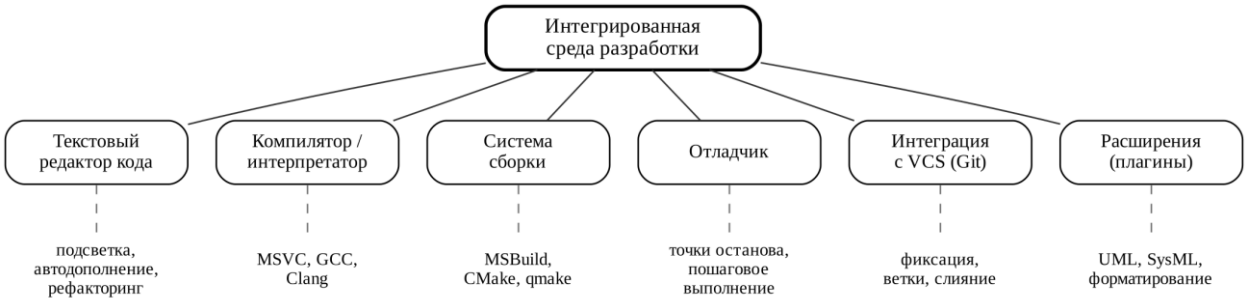


Рисунок 1.1 – Основные компоненты интегрированной среды разработки

Для работы с моделями SysML v2 и диаграммами UML в Visual Studio Code используются соответствующие расширения. В Microsoft Visual Studio их аналогами являются компонент «Конструктор классов» (Class Designer), строящий диаграмму классов по исходному коду, и расширение PlantUML, позволяющее описывать диаграммы в текстовом виде. Диаграмма классов разработанного приложения в нотации UML приведена в подразделе 2.2.

1.2 Приемы работы в интегрированной среде разработки

Основная часть работы программиста происходит в редакторе кода. Среда Visual Studio с помощью механизма IntelliSense предлагает варианты автодополнения имен классов, методов и переменных и показывает подсказки по параметрам функций. Для навигации используются переход к определению символа (F12), переключение между заголовочным файлом и файлом реализации (Ctrl+K, Ctrl+O) и поиск всех ссылок на символ (Shift+F12). К средствам рефакторинга относятся переименование символа во всем проекте (Ctrl+R, Ctrl+R) и автоматическое создание определения метода по его объявлению. Кроме того, редактор позволяет сворачивать блоки кода, форматировать документ (Ctrl+K, Ctrl+D), комментировать выделенные строки (Ctrl+K, Ctrl+C) и сразу показывает ошибки компиляции, подчеркивая их в тексте программы.

Сборка проекта выполняется системой MSBuild по описанию, хранящемуся в файле проекта с расширением vcxproj. Сборка решения запускается сочетанием клавиш Ctrl+Shift+B и включает предварительную обработку исходных текстов препроцессором, компиляцию каждого файла в объектный модуль и компоновку объектных модулей с библиотеками в исполняемый файл. Сообщения компилятора выводятся в окна «Вывод» и «Список ошибок», двойной щелчок по сообщению открывает строку с ошибкой.

Программа запускается под управлением отладчика клавишей F5 или без отладчика сочетанием Ctrl+F5. Точка останова устанавливается клавишей F9 или щелчком слева от номера строки. После останова выполнение продолжается по шагам без захода в вызываемые функции (F10), с заходом в них (F11) или до выхода из текущей функции (Shift+F11). В окнах «Локальные», «Видимые» и «Контрольные значения» отображаются значения переменных, а в окне «Стек вызовов» – цепочка вызовов, приведшая к текущей строке. Для точки останова можно задать условие, например останавливаться только при определенном значении переменной.

Работа отладчика была продемонстрирована на разработанном приложении: точка останова устанавливалась в методе PressureModel::calculate, после ввода данных выполнение останавливалось, и в окне «Локальные» проверялись вычисленные значения возраста и идеального давления.

1.3 Паттерн MVC. Активная и пассивная модель

MVC (Model-View-Controller, «модель – представление – контроллер») – архитектурный паттерн, который делит приложение на три компонента с четко разграниченными обязанностями:

– модель (Model) хранит данные приложения и реализует бизнес-логику: проверку корректности данных и вычисления; модель не зависит от того, как данные будут показаны пользователю;

– представление (View) отображает состояние модели и не содержит логики вычислений;

– контроллер (Controller) принимает действия пользователя и превращает их в вызовы методов модели.

Такое разделение делает компоненты независимыми: расчетную часть можно изменять, не затрагивая интерфейс, и наоборот, а одна модель может одновременно отображаться в нескольких представлениях.

Существуют две разновидности паттерна – с пассивной и с активной моделью. Они различаются тем, каким образом представление узнает об изменении данных.

Пассивная модель только хранит данные и предоставляет методы для их изменения, но никого не оповещает. После того как контроллер изменил модель, он сам должен сообщить представлению, что его нужно обновить, а представление затем запрашивает у модели новые данные. Схема взаимодействия компонентов при пассивной модели изображена на рисунке 1.2.

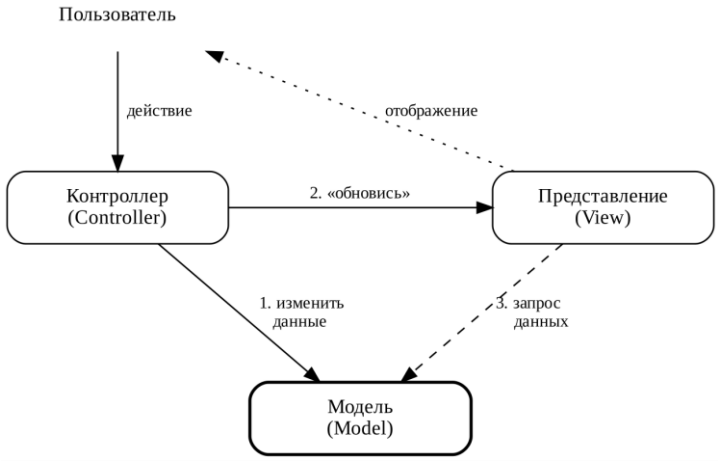


Рисунок 1.2 – Схема паттерна MVC с пассивной моделью

Активная модель построена на основе шаблона «Наблюдатель» (Observer) [2]. Модель является субъектом наблюдения: она хранит список подписчиков, а при каждом изменении своего состояния сама рассылает им уведомления. Модель при этом не знает конкретных классов представлений, ей известен только общий интерфейс наблюдателя. Представление

подписывается на модель и обновляется самостоятельно, без участия контроллера. Схема взаимодействия компонентов при активной модели изображена на рисунке 1.3.

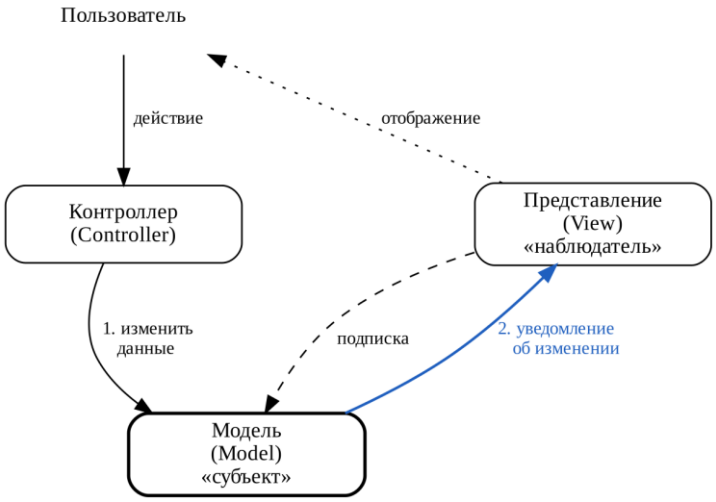


Рисунок 1.3 – Схема паттерна MVC с активной моделью

Активная модель удобна, когда данные могут измениться не только по команде пользователя, но и, например, по таймеру или при получении данных из сети, а также когда одну модель отображают несколько представлений: все они синхронизируются автоматически. По варианту задания требуется реализовать каркас MVC с активной моделью.

2 РАЗРАБОТКА ПРИЛОЖЕНИЯ

2.1 Выбор языка программирования и среды разработки

Для реализации приложения выбран объектно-ориентированный язык C++ (стандарт C++17), в качестве среды разработки – Microsoft Visual Studio 2022 Community с компилятором MSVC и системой сборки MSBuild [3]. Графический интерфейс построен непосредственно на Windows API (Win32) [4] без сторонних библиотек, поэтому готовый исполняемый файл запускается на любом компьютере с Windows и не требует установки дополнительных компонентов.

В Windows API нет готового механизма уведомлений между объектами, поэтому шаблон «Наблюдатель», лежащий в основе активной модели, реализован в проекте явно – через интерфейс IPressureObserver. Это делает механизм активной модели наглядным: в исходном тексте видно, как модель хранит список подписчиков и как рассылает им уведомления.

Проект описывается файлом решения PressureCalculator.sln и файлом проекта PressureCalculator.vcxproj. В настройках проекта включена поддержка Unicode, ключ компилятора /utf-8 для корректной обработки русского текста в исходных файлах и манифест приложения, подключающий современные элементы управления Windows (Common Controls 6). Дополнительно в проект добавлен файл CMakeLists.txt, позволяющий собрать приложение в Visual Studio Code или другой среде, поддерживающей CMake.

2.2 Структура проекта

Исходные тексты программы разложены по каталогам в соответствии с компонентами паттерна MVC: каталог src/model содержит модель, каталог src/view – классы представления, каталог src/controller – контроллер. Соответствие классов программы компонентам паттерна приведено в таблице 2.1.

Таблица 2.1 – Соответствие классов программы компонентам паттерна MVC

Компонент	Класс (файл)	Назначение
Модель	PressureModel (src/model)	Хранение введенных данных и результата, проверка данных, расчет давления, рассылка уведомлений подписчикам
Модель	PressureInput, PressureResult, IPressureObserver (src/model)	Структуры данных ввода и результата, интерфейс наблюдателя
Представление	MainWindow (src/view)	Главное окно: кнопка «Ввести данные», последние введенные данные, результат, сообщение об ошибке

Продолжение таблицы 2.1

Компонент	Класс (файл)	Назначение
Представление	InputDialog (src/view)	Модальное окно ввода даты рождения, веса и давления
Контроллер	PressureController (src/controller)	Обработка нажатия кнопки, открытие окна ввода, передача введенных данных в модель
Точка входа	main.cpp	Создание модели, представления и контроллера, их связывание, цикл обработки сообщений

Диаграмма классов приложения в нотации UML изображена на рисунке 2.1.

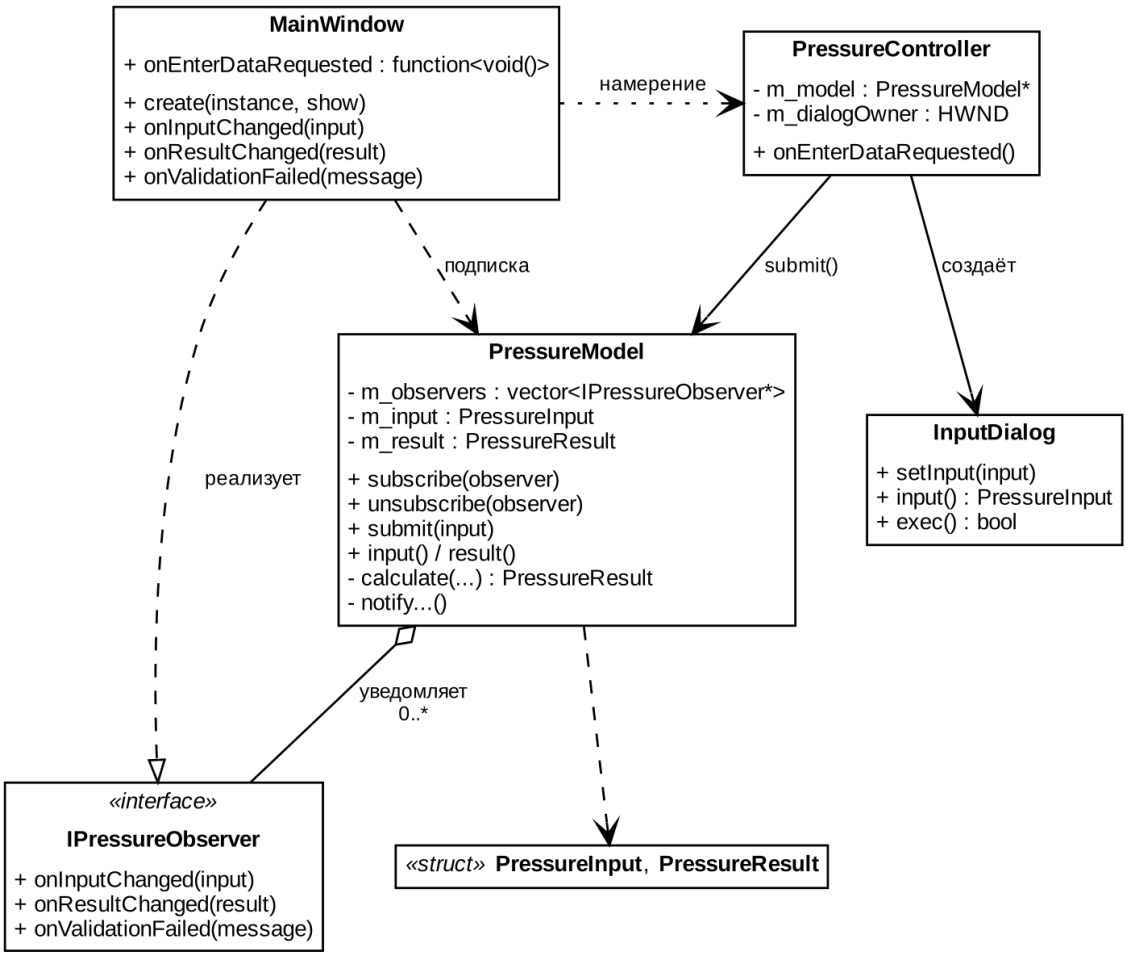


Рисунок 2.1 – Диаграмма классов приложения

2.3 Реализация каркаса MVC с активной моделью

Модель реализована классом PressureModel. Класс хранит последние введенные пользователем данные в структуре PressureInput и результат

расчета в структуре PressureResult, проверяет корректность данных и выполняет расчет. Модель написана только средствами стандартной библиотеки C++ и не подключает ни заголовочных файлов Windows API, ни файлов представления или контроллера, то есть ничего не знает о том, кто ее использует.

Активность модели обеспечивается реализацией шаблона «Наблюдатель». Модель хранит список указателей на объекты, реализующие интерфейс IPressureObserver, и предоставляет методы subscribe и unsubscribe для подписки и отписки. Интерфейс наблюдателя содержит три метода-уведомления: onInputChanged – модель приняла и запомнила введенные данные; onResultChanged – изменился результат расчета; onValidationFailed – введенные данные некорректны.

Единственной точкой входа в модель для контроллера является метод submit. Он выполняет следующие действия:

– сохраняет введенные данные и рассылает уведомление onInputChanged;

– проверяет каждое значение; при ошибке сбрасывает прежний результат, рассылает уведомления onResultChanged с пустым результатом и onValidationFailed с текстом ошибки;

– при корректных данных выполняет расчет и рассылает уведомление onResultChanged.

Введенные данные хранятся в модели в виде строк, а не чисел. Благодаря этому модель запоминает и возвращает в окно ввода ровно то, что набрал пользователь, в том числе некорректный ввод. Разбор строк и проверка значений выполняются в модели, поскольку правила вида «возраст от 18 лет» или «верхнее давление больше нижнего» относятся к логике задачи, а не к элементам интерфейса.

Главное окно MainWindow реализует интерфейс IPressureObserver, подписывается на модель в конструкторе и отписывается в деструкторе. Получив уведомление, окно само обновляет метки с последними введенными данными и результатом, а при ошибке выводит окно с сообщением. Цвет оценки давления зависит от результата: повышенное давление выделяется красным цветом, пониженное – синим, норма – зеленым.

Контроллер реализован классом PressureController. При нажатии кнопки «Ввести данные» главное окно сообщает контроллеру о намерении пользователя через функцию обратного вызова onEnterDataRequested. Контроллер создает окно ввода InputDialog, заполняет его последними данными, полученными из модели, и после подтверждения передает введенные значения в модель вызовом метода submit. Результат расчета контроллер не отображает и указателя на главное окно для его обновления не использует: обновление интерфейса выполняется по цепочке «модель – уведомление – представление».

Окно ввода InputDialog является модальным: на время его показа главное окно блокируется функцией EnableWindow, а обработка сообщений выполняется собственным циклом, в котором функция IsDialogMessage

обеспечивает переход между полями клавишей Tab, подтверждение клавишей Enter и отмену клавишей Esc. Дата рождения вводится тремя отдельными текстовыми полями (день, месяц, год), поскольку по условию задания использовать стандартный элемент управления для ввода даты нельзя. Корректность даты проверяется в модели, что позволяет отсеять несуществующие даты вида 31 февраля.

2.4 Алгоритм расчета нормального давления

Расчет выполняется в методе calculate модели и состоит из четырех шагов.

Шаг 1. По дате рождения и текущей дате определяется возраст человека в полных годах.

Шаг 2. По эмпирическим формулам вычисляется «идеальное» давление для возраста и веса:

$$САД = 109 + 0{,}5 \cdot В + 0{,}1 \cdot М,$$
$$ДАД = 63 + 0{,}1 \cdot В + 0{,}15 \cdot М,$$

где САД – верхнее (систолическое) давление, мм рт. ст.; ДАД – нижнее (диастолическое) давление, мм рт. ст.; В – возраст, лет; М – масса тела, кг. Результаты округляются до целого.

Шаг 3. Определяется отклонение текущего давления от идеального отдельно для верхнего и нижнего значений.

Шаг 4. Выставляется оценка текущего давления. Давление считается повышенным, если верхнее давление не менее 140 мм рт. ст. или нижнее не менее 90 мм рт. ст. (границы артериальной гипертонии по данным ВОЗ [5]), либо если верхнее давление превышает идеальное более чем на 15 мм рт. ст., а нижнее – более чем на 10 мм рт. ст. Давление считается пониженным, если верхнее давление меньше 90 мм рт. ст. или нижнее меньше 60 мм рт. ст., либо если отклонение от идеального в меньшую сторону превышает те же допуски. В остальных случаях давление оценивается как норма. Повышенное давление проверяется первым как более опасное состояние. К оценке добавляется краткая рекомендация.

Например, для человека 21 года весом 68 кг идеальное давление составляет $109 + 0{,}5 \cdot 21 + 0{,}1 \cdot 68 \approx 126$ и $63 + 0{,}1 \cdot 21 + 0{,}15 \cdot 68 \approx 75$, то есть 126/75 мм рт. ст. Текущее давление 125/82 мм рт. ст. отличается от идеального на –1 и +7 мм рт. ст. и оценивается как норма.

Проверке подлежат все введенные значения: день от 1 до 31, месяц от 1 до 12, год от 1900 до текущего, существование даты в календаре, отсутствие даты в будущем, возраст от 18 до 120 лет (формулы рассчитаны на взрослых), вес от 30 до 300 кг (допускается дробное значение с точкой или запятой), верхнее давление от 60 до 260 мм рт. ст., нижнее – от 30 до 160 мм рт. ст., причем верхнее давление должно быть больше нижнего.

3 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

После запуска программы отображается главное окно с кнопкой «Ввести данные». Последние введенные данные и результат расчета еще отсутствуют, вместо них выведены прочерки. Вид главного окна после запуска изображен на рисунке 3.1.

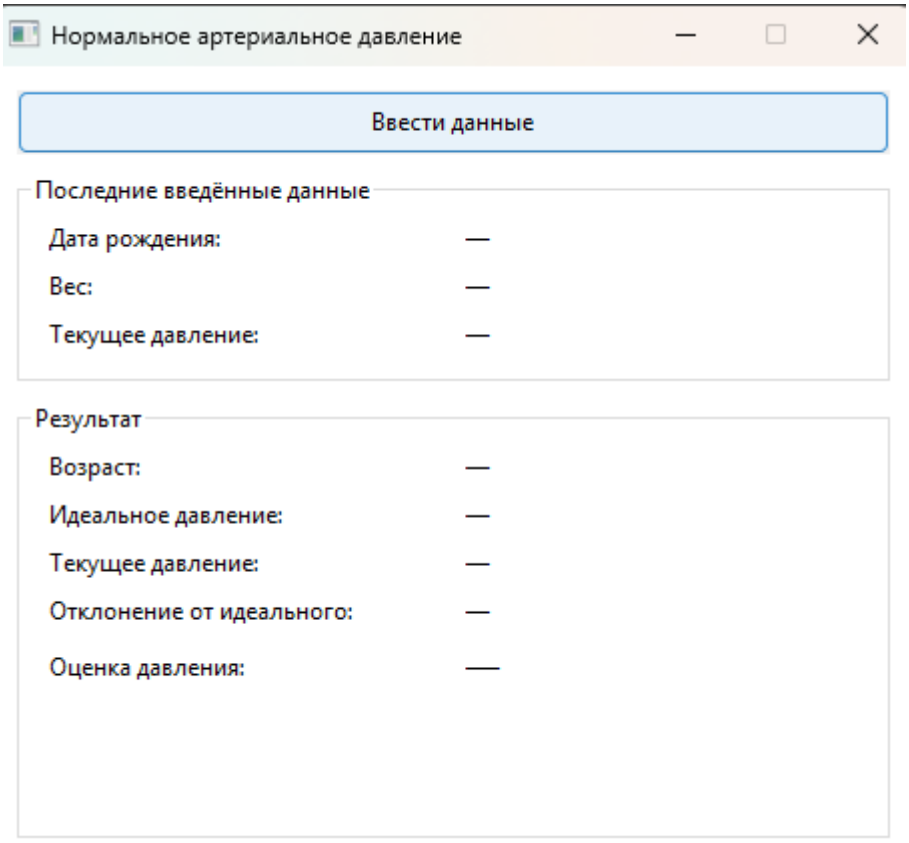


Рисунок 3.1 – Главное окно программы после запуска

По нажатию на кнопку «Ввести данные» открывается модальное окно ввода. Заполненное окно ввода изображено на рисунке 3.2.

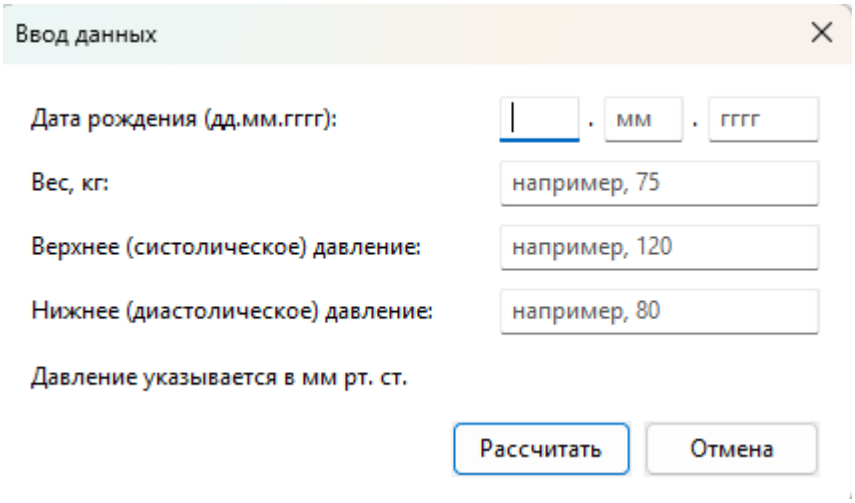


Рисунок 3.2 – Окно ввода исходных данных

После нажатия кнопки «Рассчитать» модель принимает данные, выполняет расчет и рассылает уведомления, по которым главное окно отображает введенные данные и результат. Главное окно с результатом расчета изображено на рисунке 3.3.

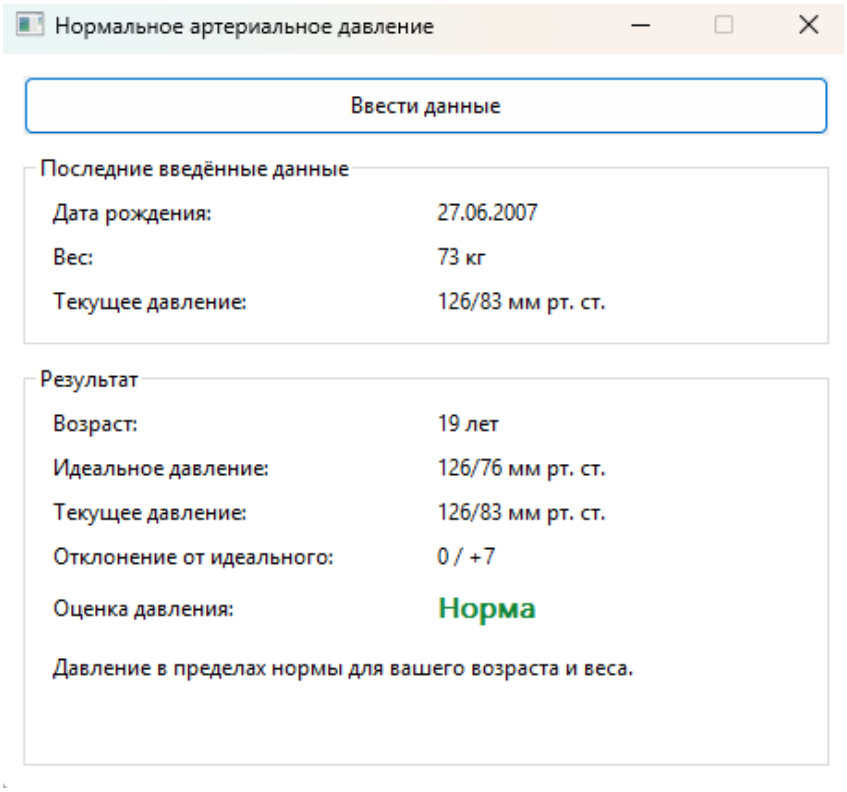


Рисунок 3.3 – Главное окно с результатом расчета (норма)

При повторном нажатии на кнопку «Ввести данные» окно ввода заполняется последними введенными значениями, полученными из модели. Восстановленные данные изображены на рисунке 3.4.

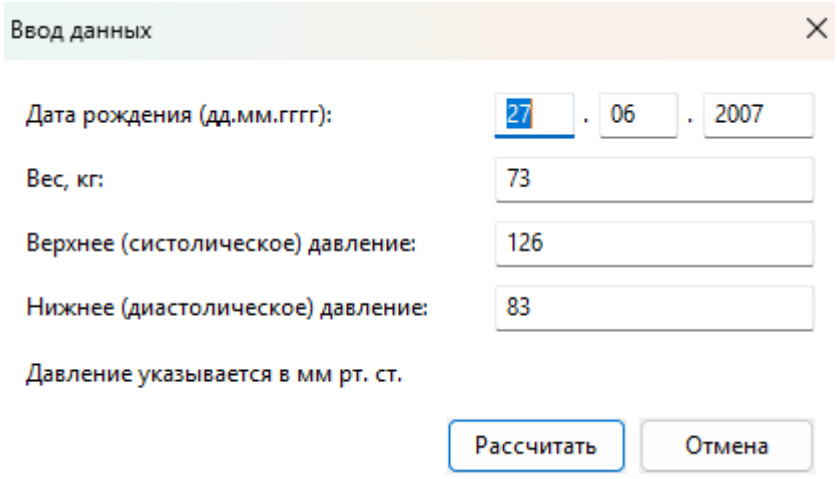


Рисунок 3.4 – Восстановление последних введенных данных

Результаты расчета для повышенного и пониженного давления изображены на рисунках 3.5 и 3.6 соответственно.

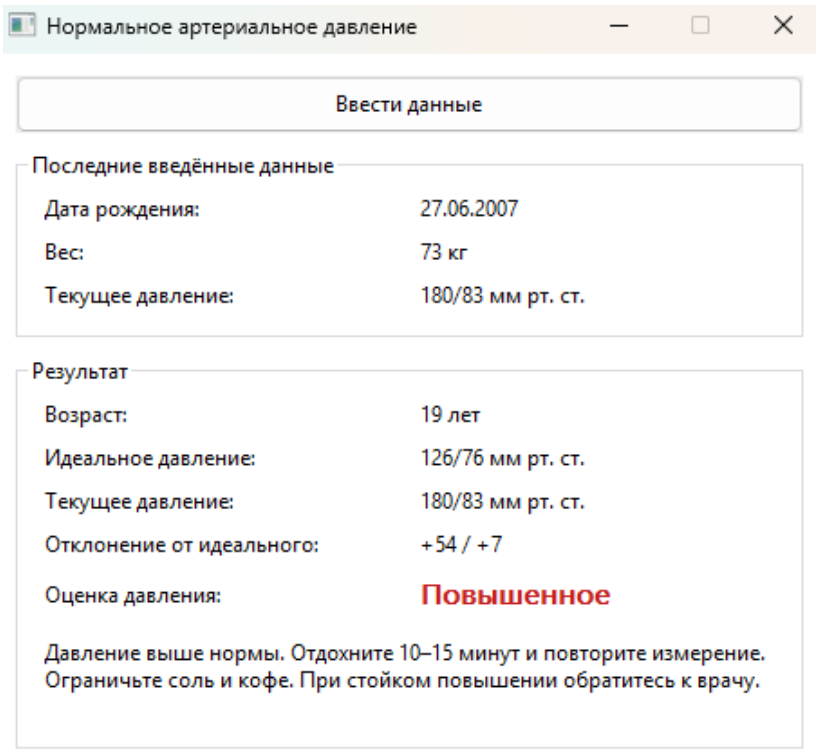


Рисунок 3.5 – Результат расчета при повышенном давлении

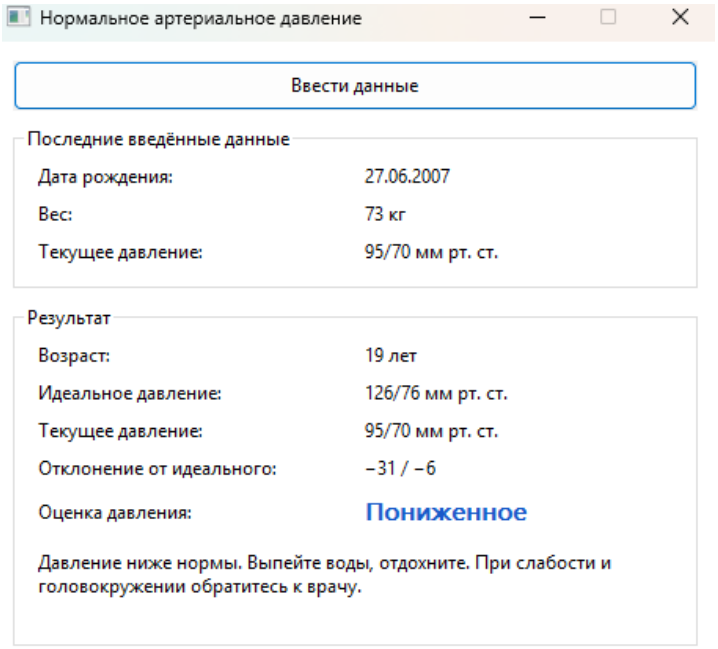


Рисунок 3.6 – Результат расчета при пониженном давлении

Если введенные данные некорректны, модель сбрасывает устаревший результат и формирует уведомление об ошибке, по которому главное окно выводит сообщение с описанием причины. Сообщение об ошибке при вводе несуществующей даты 31.02.2005 изображено на рисунке 3.7.

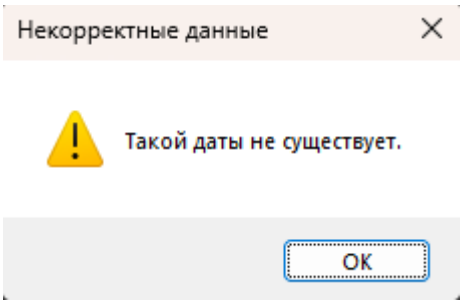


Рисунок 3.7 – Сообщение о некорректных данных

Все элементы задания по варианту 10 выполнены: главная форма содержит кнопку «Ввести данные», дата рождения, вес и давление вводятся в отдельном окне без стандартного элемента ввода даты, последние введенные данные запоминаются и восстанавливаются при повторном вводе, на главной форме отображаются идеальное давление и оценка текущего давления, при некорректных данных выводится сообщение об ошибке, каркас MVC построен с активной моделью.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены назначение, состав и основные приемы работы с интегрированной средой разработки программного обеспечения, а также архитектурный паттерн MVC и различия между его реализациями с пассивной и активной моделью.

В качестве среды разработки выбрана Microsoft Visual Studio 2022, в качестве языка программирования – C++ с использованием Windows API. В выбранной среде создан проект и разработано приложение с графическим интерфейсом пользователя, реализующее задание варианта 10 – утилиту для информирования пользователя о нормальном артериальном давлении. Изучены средства сборки и отладки: точки останова, пошаговое выполнение и просмотр значений переменных.

Каркас приложения построен в соответствии с паттерном MVC с активной моделью: класс PressureModel хранит данные, выполняет проверку и расчет и сам оповещает подписчиков через интерфейс IPressureObserver; классы MainWindow и InputDialog образуют представление, причем главное окно подписывается на модель напрямую; класс PressureController обрабатывает действия пользователя и не участвует в обновлении интерфейса. Такое разделение делает логику расчета независимой от способа отображения данных.

Работоспособность приложения подтверждена: реализованы запоминание и восстановление последних введенных данных, проверка корректности ввода с выводом сообщения об ошибке, расчет идеального давления и оценка текущего давления с отображением результата на главной форме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Жизненный цикл разработки программного обеспечения. Лабораторная работа №1. Интегрированная среда разработки ПО [Электронный ресурс] : задание к лабораторной работе. – Минск : БГУИР, 2025.

[2] Гамма, Э. Паттерны объектно-ориентированного проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – СПб. : Питер, 2021. – 448 с.

[3] Документация по Visual Studio [Электронный ресурс] / Microsoft. – Режим доступа : https://learn.microsoft.com/ru-ru/visualstudio/.

[4] Документация по Win32 API [Электронный ресурс] / Microsoft. – Режим доступа : https://learn.microsoft.com/ru-ru/windows/win32/.

[5] Гипертония : информационный бюллетень [Электронный ресурс] / Всемирная организация здравоохранения. – Режим доступа : https://www.who.int/ru/news-room/fact-sheets/detail/hypertension.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг кода

Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(PressureCalculator LANGUAGES CXX RC)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(PressureCalculator WIN32
    src/main.cpp
    src/model/pressuremodel.cpp
    src/view/mainwindow.cpp
    src/view/inputdialog.cpp
    src/controller/pressurecontroller.cpp
)

target_include_directories(PressureCalculator PRIVATE src)
target_compile_definitions(PressureCalculator PRIVATE UNICODE _UNICODE)
target_link_libraries(PressureCalculator PRIVATE comctl32)

if(MSVC)
    target_compile_options(PressureCalculator PRIVATE /utf-8 /W4)
    target_sources(PressureCalculator PRIVATE app.manifest)
else()
    target_sources(PressureCalculator PRIVATE app_mingw.rc)
    target_link_options(PressureCalculator PRIVATE -municode -static)
endif()
```

Файл src/model/pressuremodel.h

```
#ifndef PRESSUREMODEL_H
#define PRESSUREMODEL_H

#include <string>
#include <vector>

// Данные, введённые пользователем в окне ввода.
// Хранятся строками: так модель может запомнить и вернуть в окно
// ровно то, что набрал пользователь, даже если это некорректный ввод.
struct PressureInput
{
    std::wstring day;        // день рождения
    std::wstring month;      // месяц рождения
    std::wstring year;       // год рождения
    std::wstring weight;     // вес, кг
    std::wstring systolic;   // верхнее (систолическое) давление, мм рт. ст.
    std::wstring diastolic;  // нижнее (диастолическое) давление, мм рт. ст.
};

// Оценка текущего давления.
enum class PressureLevel
{
    Unknown,  // расчёт не выполнялся
    Low,      // пониженное
    Normal,   // норма
```

```
    High      // повышенное
};

// Результат расчёта.
struct PressureResult
{
    bool          valid          = false;
    int           age            = 0;    // полных лет
    double        weight         = 0.0;  // кг
    int           idealSystolic  = 0;    // «идеальное» верхнее давление
    int           idealDiastolic = 0;    // «идеальное» нижнее давление
    int           systolic       = 0;    // текущее верхнее давление
    int           diastolic      = 0;    // текущее нижнее давление
    int           deltaSystolic  = 0;    // отклонение верхнего от идеального
    int           deltaDiastolic = 0;    // отклонение нижнего от идеального
    PressureLevel level          = PressureLevel::Unknown;
    std::wstring  verdict;               // «Повышенное» / «Пониженное» /
«Норма»
    std::wstring  advice;                // рекомендация
};

// Календарная дата (модель не зависит от WinAPI, поэтому своя структура).
struct SimpleDate
{
    int day   = 1;
    int month = 1;
    int year  = 2000;
};

// Интерфейс наблюдателя (шаблон Observer).
// Его реализует любое представление, которое хочет узнавать
// об изменениях модели.
class IPressureObserver
{
public:
    virtual ~IPressureObserver() = default;

    virtual void onInputChanged(const PressureInput &input) = 0;
    virtual void onResultChanged(const PressureResult &result) = 0;
    virtual void onValidationFailed(const std::wstring &message) = 0;
};

// АКТИВНАЯ модель: хранит данные, проверяет их, выполняет расчёт и при любом
// изменении своего состояния сама рассылает уведомления подписчикам.
// О конкретных окнах и о контроллере модель ничего не знает.
class PressureModel
{
public:
    PressureModel() = default;

    // Подписка / отписка наблюдателей.
    void subscribe(IPressureObserver *observer);
    void unsubscribe(IPressureObserver *observer);

    const PressureInput  &input()  const { return m_input; }
    const PressureResult &result() const { return m_result; }

    // Единственная точка входа для контроллера.
    void submit(const PressureInput &input);
```

```
    // Текущая дата (по умолчанию — системная). Можно подменить для тестов.
    void setToday(const SimpleDate &today);
    SimpleDate today() const;

    // Формулы расчёта «идеального» давления.
    static int idealSystolic(int age, double weight);
    static int idealDiastolic(int age, double weight);

private:
    PressureResult calculate(int age, double weight,
                             int systolic, int diastolic) const;

    void notifyInputChanged();
    void notifyResultChanged();
    void notifyValidationFailed(const std::wstring &message);

    std::vector<IPressureObserver *> m_observers;

    PressureInput  m_input;
    PressureResult m_result;

    bool       m_hasFixedToday = false;
    SimpleDate m_fixedToday;
};

#endif // PRESSUREMODEL_H
```

Файл src/model/pressuremodel.cpp

```
#include "model/pressuremodel.h"

#include <algorithm>
#include <cmath>
#include <cstdlib>
#include <ctime>
#include <cwctype>

namespace {

// Ограничения на вводимые значения.
constexpr int    kMinYear      = 1900;
constexpr int    kMinAge       = 18;    // формула рассчитана на взрослых
constexpr int    kMaxAge       = 120;
constexpr double kMinWeight    = 30.0;
constexpr double kMaxWeight    = 300.0;
constexpr int    kMinSystolic  = 60;
constexpr int    kMaxSystolic  = 260;
constexpr int    kMinDiastolic = 30;
constexpr int    kMaxDiastolic = 160;

// Допустимое отклонение текущего давления от «идеального», мм рт. ст.
constexpr int kSystolicTolerance  = 15;
constexpr int kDiastolicTolerance = 10;

// Общепринятые границы гипертонии и гипотонии, мм рт. ст.
constexpr int kHighSystolic  = 140;
constexpr int kHighDiastolic = 90;
constexpr int kLowSystolic   = 90;
```

```
constexpr int kLowDiastolic  = 60;

// Удаление пробелов по краям строки.
std::wstring trimmed(const std::wstring &text)
{
    const auto begin = std::find_if_not(text.begin(), text.end(),
                                        [](wchar_t c) { return
std::iswspace(c); });
    const auto end = std::find_if_not(text.rbegin(), text.rend(),
                                      [](wchar_t c) { return
std::iswspace(c); }).base();
    return begin < end ? std::wstring(begin, end) : std::wstring();
}

// Разбор целого числа с проверкой диапазона.
bool parseInt(const std::wstring &text, int minValue, int maxValue, int &out)
{
    const std::wstring s = trimmed(text);
    if (s.empty() || s.size() > 9)
        return false;
    for (wchar_t c : s)
        if (c < L'0' || c > L'9')
            return false;

    const long value = std::wcstol(s.c_str(), nullptr, 10);
    if (value < minValue || value > maxValue)
        return false;
    out = static_cast<int>(value);
    return true;
}

// Разбор вещественного числа (запятая допускается как разделитель).
bool parseDouble(const std::wstring &text, double minValue, double maxValue,
double &out)
{
    std::wstring s = trimmed(text);
    std::replace(s.begin(), s.end(), L',', L'.');
    if (s.empty())
        return false;

    wchar_t *end = nullptr;
    const double value = std::wcstod(s.c_str(), &end);
    if (end == s.c_str() || *end != L'\0' || !std::isfinite(value))
        return false;
    if (value < minValue || value > maxValue)
        return false;
    out = value;
    return true;
}

bool isLeapYear(int year)
{
    return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
}

int daysInMonth(int month, int year)
{
    static const int days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31 };
    if (month == 2 && isLeapYear(year))
        return 29;
```

```
    return days[month - 1];
}

// Сравнение дат: <0, 0, >0.
int compareDates(const SimpleDate &a, const SimpleDate &b)
{
    if (a.year != b.year)   return a.year  - b.year;
    if (a.month != b.month) return a.month - b.month;
    return a.day - b.day;
}

// Количество полных лет на дату today.
int fullYears(const SimpleDate &birth, const SimpleDate &today)
{
    int age = today.year - birth.year;
    if (today.month < birth.month
        || (today.month == birth.month && today.day < birth.day))
        --age;
    return age;
}

} // namespace

// ----------------------------------------------------------------------------
-
// Подписка наблюдателей
// ----------------------------------------------------------------------------
-

void PressureModel::subscribe(IPressureObserver *observer)
{
    if (observer && std::find(m_observers.begin(), m_observers.end(),
observer) == m_observers.end())
        m_observers.push_back(observer);
}

void PressureModel::unsubscribe(IPressureObserver *observer)
{
    m_observers.erase(std::remove(m_observers.begin(), m_observers.end(),
observer),
                      m_observers.end());
}

void PressureModel::notifyInputChanged()
{
    const auto observers = m_observers; // копия: подписчик может отписаться
в обработчике
    for (IPressureObserver *o : observers)
        o->onInputChanged(m_input);
}

void PressureModel::notifyResultChanged()
{
    const auto observers = m_observers;
    for (IPressureObserver *o : observers)
        o->onResultChanged(m_result);
}

void PressureModel::notifyValidationFailed(const std::wstring &message)
{
```

```
    const auto observers = m_observers;
    for (IPressureObserver *o : observers)
        o->onValidationFailed(message);
}

// ------------------------------------------------------------------------
-
// Текущая дата
// ------------------------------------------------------------------------
-

void PressureModel::setToday(const SimpleDate &today)
{
    m_fixedToday = today;
    m_hasFixedToday = true;
}

SimpleDate PressureModel::today() const
{
    if (m_hasFixedToday)
        return m_fixedToday;

    const std::time_t now = std::time(nullptr);
    std::tm local{};
#ifdef _WIN32
    localtime_s(&local, &now);
#else
    localtime_r(&now, &local);
#endif
    return SimpleDate{ local.tm_mday, local.tm_mon + 1, local.tm_year + 1900
};
}

// ------------------------------------------------------------------------
-
// Формулы
// ------------------------------------------------------------------------
-

// Верхнее (систолическое): 109 + 0,5 * возраст + 0,1 * вес
int PressureModel::idealSystolic(int age, double weight)
{
    return static_cast<int>(std::lround(109.0 + 0.5 * age + 0.1 * weight));
}

// Нижнее (диастолическое): 63 + 0,1 * возраст + 0,15 * вес
int PressureModel::idealDiastolic(int age, double weight)
{
    return static_cast<int>(std::lround(63.0 + 0.1 * age + 0.15 * weight));
}

// ------------------------------------------------------------------------
-
// Приём данных от контроллера
// ------------------------------------------------------------------------
-

void PressureModel::submit(const PressureInput &input)
{
    // 1. Введённые данные запоминаются в любом случае, даже некорректные:
```

```
    //    при повторном вводе окно будет ими заполнено.
    m_input = input;
    notifyInputChanged();

    // 2. Проверка корректности. При ошибке прежний результат сбрасывается,
    //    подписчики получают пустой результат и сообщение об ошибке.
    auto fail = [this](const std::wstring &message) {
        m_result = PressureResult{};
        notifyResultChanged();
        notifyValidationFailed(message);
    };

    const SimpleDate now = today();
    int day = 0, month = 0, year = 0;

    if (!parseInt(input.day, 1, 31, day)) {
        fail(L"День рождения должен быть целым числом от 1 до 31.");
        return;
    }
    if (!parseInt(input.month, 1, 12, month)) {
        fail(L"Месяц рождения должен быть целым числом от 1 до 12.");
        return;
    }
    if (!parseInt(input.year, kMinYear, now.year, year)) {
        fail(L"Год рождения должен быть целым числом от " +
std::to_wstring(kMinYear)
             + L" до " + std::to_wstring(now.year) + L".");
        return;
    }
    if (day > daysInMonth(month, year)) {
        fail(L"Такой даты не существует в календаре.");
        return;
    }

    const SimpleDate birth{ day, month, year };
    if (compareDates(birth, now) > 0) {
        fail(L"Дата рождения не может быть в будущем.");
        return;
    }

    const int age = fullYears(birth, now);
    if (age < kMinAge || age > kMaxAge) {
        fail(L"Расчёт выполняется для взрослых: возраст должен быть от "
             + std::to_wstring(kMinAge) + L" до " + std::to_wstring(kMaxAge)
             + L" лет (сейчас " + std::to_wstring(age) + L").");
        return;
    }

    double weight = 0.0;
    if (!parseDouble(input.weight, kMinWeight, kMaxWeight, weight)) {
        fail(L"Вес должен быть числом от 30 до 300 кг.");
        return;
    }

    int systolic = 0, diastolic = 0;
    if (!parseInt(input.systolic, kMinSystolic, kMaxSystolic, systolic)) {
        fail(L"Верхнее давление должно быть целым числом от "
             + std::to_wstring(kMinSystolic) + L" до " +
std::to_wstring(kMaxSystolic)
             + L" мм рт. ст.");
        return;
```

```
    }
    if (!parseInt(input.diastolic, kMinDiastolic, kMaxDiastolic, diastolic))
{
        fail(L"Нижнее давление должно быть целым числом от "
             + std::to_wstring(kMinDiastolic) + L" до " +
std::to_wstring(kMaxDiastolic)
             + L" мм рт. ст.");
        return;
    }
    if (systolic <= diastolic) {
        fail(L"Верхнее давление должно быть больше нижнего.");
        return;
    }

    // 3. Расчёт и уведомление подписчиков.
    m_result = calculate(age, weight, systolic, diastolic);
    notifyResultChanged();
}

// ----------------------------------------------------------------------------
-
// Расчёт
// ----------------------------------------------------------------------------
-

PressureResult PressureModel::calculate(int age, double weight,
                                        int systolic, int diastolic) const
{
    PressureResult r;
    r.valid  = true;
    r.age    = age;
    r.weight = weight;

    // Шаг 1. «Идеальное» давление для возраста и веса.
    r.idealSystolic  = idealSystolic(age, weight);
    r.idealDiastolic = idealDiastolic(age, weight);

    // Шаг 2. Отклонение текущего давления от идеального.
    r.systolic       = systolic;
    r.diastolic      = diastolic;
    r.deltaSystolic  = systolic  - r.idealSystolic;
    r.deltaDiastolic = diastolic - r.idealDiastolic;

    // Шаг 3. Оценка. Повышенное давление проверяется первым,
    // так как оно опаснее (например, при изолированной систолической
гипертонии).
    const bool high = systolic >= kHighSystolic || diastolic >=
kHighDiastolic
                   || r.deltaSystolic  >  kSystolicTolerance
                   || r.deltaDiastolic >  kDiastolicTolerance;
    const bool low  = systolic <  kLowSystolic  || diastolic <  kLowDiastolic
                   || r.deltaSystolic  < -kSystolicTolerance
                   || r.deltaDiastolic < -kDiastolicTolerance;

    if (high) {
        r.level   = PressureLevel::High;
        r.verdict = L"Повышенное";
        r.advice  = L"Давление выше нормы. Отдохните 10–15 минут и повторите
измерение. "
                    L"Ограничьте соль и кофе. При стойком повышении
обратитесь к врачу.";
```

```
    } else if (low) {
        r.level   = PressureLevel::Low;
        r.verdict = L"Пониженное";
        r.advice  = L"Давление ниже нормы. Выпейте воды, отдохните. "
                    L"При слабости и головокружении обратитесь к врачу.";
    } else {
        r.level   = PressureLevel::Normal;
        r.verdict = L"Норма";
        r.advice  = L"Давление в пределах нормы для вашего возраста и веса.";
    }
    return r;
}
```

Файл src/view/uihelpers.h

```
#ifndef UIHELPERS_H
#define UIHELPERS_H

// Вспомогательные функции для построения интерфейса на WinAPI.

#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>

#include <string>

namespace ui {

// Масштабирование размеров под текущий DPI экрана (96 DPI = 100 %).
inline int scale(int value)
{
    static int dpi = [] {
        HDC dc = GetDC(nullptr);
        const int d = GetDeviceCaps(dc, LOGPIXELSY);
        ReleaseDC(nullptr, dc);
        return d > 0 ? d : 96;
    }();
    return MulDiv(value, dpi, 96);
}

// Системный шрифт интерфейса (Segoe UI). bold и sizeFactor — для выделенных
меток.
inline HFONT createUiFont(bool bold = false, double sizeFactor = 1.0)
{
    NONCLIENTMETRICSW metrics{};
    metrics.cbSize = sizeof(metrics);
    SystemParametersInfoW(SPI_GETNONCLIENTMETRICS, sizeof(metrics), &metrics,
0);

    LOGFONTW font = metrics.lfMessageFont;
    font.lfHeight = static_cast<LONG>(font.lfHeight * sizeFactor);
    if (bold)
        font.lfWeight = FW_SEMIBOLD;
    return CreateFontIndirectW(&font);
}

// Текст поля ввода.
inline std::wstring windowText(HWND hwnd)
{
```

```
    const int length = GetWindowTextLengthW(hwnd);
    std::wstring text(static_cast<size_t>(length) + 1, L'\0');
    GetWindowTextW(hwnd, &text[0], length + 1);
    text.resize(static_cast<size_t>(length));
    return text;
}

// Создание дочернего элемента управления.
inline HWND createChild(HWND parent, const wchar_t *className, const wchar_t
*text,
                        DWORD style, int x, int y, int w, int h,
                        int id = 0, DWORD exStyle = 0)
{
    return CreateWindowExW(exStyle, className, text,
                           WS_CHILD | WS_VISIBLE | style,
                           scale(x), scale(y), scale(w), scale(h),
                           parent,
reinterpret_cast<HMENU>(static_cast<INT_PTR>(id)),
                           GetModuleHandleW(nullptr), nullptr);
}

// Установка шрифта всем дочерним окнам.
inline void applyFontToChildren(HWND parent, HFONT font)
{
    EnumChildWindows(parent, [](HWND child, LPARAM lParam) -> BOOL {
        SendMessageW(child, WM_SETFONT, static_cast<WPARAM>(lParam), TRUE);
        return TRUE;
    }, reinterpret_cast<LPARAM>(font));
}

const wchar_t kEmpty[] = L"—"; // длинное тире — «нет данных»

} // namespace ui

#endif // UIHELPERS_H
```

Файл src/view/mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>

#include <functional>

#include "model/pressuremodel.h"

// Главное окно. В MVC с активной моделью представление само подписывается
// на модель (реализует IPressureObserver) и обновляет себя при уведомлениях,
// без участия контроллера.
class MainWindow : public IPressureObserver
{
public:
    explicit MainWindow(PressureModel *model);
    ~MainWindow() override;
```

```
    MainWindow(const MainWindow &) = delete;
    MainWindow &operator=(const MainWindow &) = delete;

    bool create(HINSTANCE instance, int showCommand);
    HWND hwnd() const { return m_hwnd; }

    // Намерение пользователя «ввести данные». Обрабатывает контроллер.
    std::function<void()> onEnterDataRequested;

    // IPressureObserver
    void onInputChanged(const PressureInput &input) override;
    void onResultChanged(const PressureResult &result) override;
    void onValidationFailed(const std::wstring &message) override;

private:
    static LRESULT CALLBACK windowProc(HWND hwnd, UINT message, WPARAM
wParam, LPARAM lParam);
    LRESULT handleMessage(UINT message, WPARAM wParam, LPARAM lParam);

    void createControls();
    void clearResult();

    PressureModel *m_model;
    HWND m_hwnd = nullptr;

    HFONT m_font        = nullptr;
    HFONT m_verdictFont = nullptr;

    // Последние введённые данные
    HWND m_birthDate = nullptr;
    HWND m_weight    = nullptr;
    HWND m_pressure  = nullptr;

    // Результат расчёта
    HWND m_age           = nullptr;
    HWND m_idealPressure = nullptr;
    HWND m_current       = nullptr;
    HWND m_deviation     = nullptr;
    HWND m_verdict       = nullptr;
    HWND m_advice        = nullptr;

    PressureLevel m_level = PressureLevel::Unknown;
};

#endif // MAINWINDOW_H
```

Файл src/view/mainwindow.cpp

```
#include "view/mainwindow.h"

#include "view/uihelpers.h"

namespace {

const wchar_t kClassName[] = L"PressureCalculatorMainWindow";
constexpr int kEnterButtonId = 101;
```

```
// Склонение слова «год» после числа: 1 год, 2 года, 5 лет.
std::wstring yearsWord(int n)
{
    const int mod100 = n % 100;
    const int mod10  = n % 10;
    if (mod100 >= 11 && mod100 <= 14) return L"лет";
    if (mod10 == 1)                   return L"год";
    if (mod10 >= 2 && mod10 <= 4)     return L"года";
    return L"лет";
}

// Число со знаком: +5, -3, 0.
std::wstring signedNumber(int value)
{
    if (value > 0) return L"+" + std::to_wstring(value);
    if (value < 0) return L"-" + std::to_wstring(-value);
    return L"0";
}

std::wstring pressureText(int systolic, int diastolic)
{
    return std::to_wstring(systolic) + L"/" + std::to_wstring(diastolic) + L"
мм рт. ст.";
}

} // namespace

MainWindow::MainWindow(PressureModel *model)
    : m_model(model)
{
    // Подписка представления на активную модель.
    m_model->subscribe(this);
}

MainWindow::~MainWindow()
{
    m_model->unsubscribe(this);
    if (m_font)        DeleteObject(m_font);
    if (m_verdictFont) DeleteObject(m_verdictFont);
}

bool MainWindow::create(HINSTANCE instance, int showCommand)
{
    WNDCLASSEXW wc{};
    wc.cbSize        = sizeof(wc);
    wc.lpfnWndProc   = &MainWindow::windowProc;
    wc.hInstance     = instance;
    wc.hCursor       = LoadCursorW(nullptr, IDC_ARROW);
    wc.hIcon         = LoadIconW(nullptr, IDI_APPLICATION);
    wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_WINDOW + 1);
    wc.lpszClassName = kClassName;
    RegisterClassExW(&wc);

    const DWORD style = WS_OVERLAPPED | WS_CAPTION | WS_SYSMENU |
WS_MINIMIZEBOX;
    RECT rect{ 0, 0, ui::scale(460), ui::scale(398) };
    AdjustWindowRect(&rect, style, FALSE);

    m_hwnd = CreateWindowExW(0, kClassName, L"Нормальное артериальное
давление", style,
```

```
                              CW_USEDEFAULT, CW_USEDEFAULT,
                              rect.right - rect.left, rect.bottom - rect.top,
                              nullptr, nullptr, instance, this);
    if (!m_hwnd)
        return false;

    ShowWindow(m_hwnd, showCommand);
    UpdateWindow(m_hwnd);
    return true;
}

void MainWindow::createControls()
{
    m_font        = ui::createUiFont();
    m_verdictFont = ui::createUiFont(true, 1.35);

    ui::createChild(m_hwnd, L"BUTTON", L"Ввести данные",
                    WS_TABSTOP | BS_PUSHBUTTON, 12, 12, 436, 32,
kEnterButtonId);

    // Группа «Последние введённые данные»
    ui::createChild(m_hwnd, L"BUTTON", L"Последние введённые данные",
                    BS_GROUPBOX, 12, 54, 436, 104);
    ui::createChild(m_hwnd, L"STATIC", L"Дата рождения:",  0, 28, 78, 200,
20);
    ui::createChild(m_hwnd, L"STATIC", L"Вес:",            0, 28, 102, 200,
20);
    ui::createChild(m_hwnd, L"STATIC", L"Текущее давление:", 0, 28, 126, 200,
20);
    m_birthDate = ui::createChild(m_hwnd, L"STATIC", ui::kEmpty, 0, 236, 78,
200, 20);
    m_weight    = ui::createChild(m_hwnd, L"STATIC", ui::kEmpty, 0, 236, 102,
200, 20);
    m_pressure  = ui::createChild(m_hwnd, L"STATIC", ui::kEmpty, 0, 236, 126,
200, 20);

    // Группа «Результат»
    ui::createChild(m_hwnd, L"BUTTON", L"Результат",
                    BS_GROUPBOX, 12, 168, 436, 218);
    ui::createChild(m_hwnd, L"STATIC", L"Возраст:",                  0, 28,
192, 200, 20);
    ui::createChild(m_hwnd, L"STATIC", L"Идеальное давление:",       0, 28,
216, 200, 20);
    ui::createChild(m_hwnd, L"STATIC", L"Текущее давление:",         0, 28,
240, 200, 20);
    ui::createChild(m_hwnd, L"STATIC", L"Отклонение от идеального:", 0, 28,
264, 200, 20);
    ui::createChild(m_hwnd, L"STATIC", L"Оценка давления:",          0, 28,
292, 200, 20);
    m_age           = ui::createChild(m_hwnd, L"STATIC", ui::kEmpty, 0, 236,
192, 200, 20);
    m_idealPressure = ui::createChild(m_hwnd, L"STATIC", ui::kEmpty, 0, 236,
216, 200, 20);
    m_current       = ui::createChild(m_hwnd, L"STATIC", ui::kEmpty, 0, 236,
240, 200, 20);
    m_deviation     = ui::createChild(m_hwnd, L"STATIC", ui::kEmpty, 0, 236,
264, 200, 20);
    m_verdict       = ui::createChild(m_hwnd, L"STATIC", ui::kEmpty, 0, 236,
288, 200, 26);
    m_advice        = ui::createChild(m_hwnd, L"STATIC", L"", 0, 28, 324,
404, 54);
```

```
    ui::applyFontToChildren(m_hwnd, m_font);
    SendMessageW(m_verdict, WM_SETFONT,
reinterpret_cast<WPARAM>(m_verdictFont), TRUE);
}

// ------------------------------------------------------------------------
-
// Обработчики уведомлений активной модели
// ------------------------------------------------------------------------
-

void MainWindow::onInputChanged(const PressureInput &input)
{
    SetWindowTextW(m_birthDate, (input.day + L"." + input.month + L"." +
input.year).c_str());
    SetWindowTextW(m_weight,    (input.weight + L" кг").c_str());
    SetWindowTextW(m_pressure,  (input.systolic + L"/" + input.diastolic + L"
мм рт. ст.").c_str());
}

void MainWindow::onResultChanged(const PressureResult &result)
{
    if (!result.valid) {
        // Прошлый результат перестал соответствовать введённым данным.
        clearResult();
        return;
    }

    m_level = result.level;
    SetWindowTextW(m_age, (std::to_wstring(result.age) + L" " +
yearsWord(result.age)).c_str());
    SetWindowTextW(m_idealPressure, pressureText(result.idealSystolic,
result.idealDiastolic).c_str());
    SetWindowTextW(m_current, pressureText(result.systolic,
result.diastolic).c_str());
    SetWindowTextW(m_deviation, (signedNumber(result.deltaSystolic) + L" / "
                                 +
signedNumber(result.deltaDiastolic)).c_str());
    SetWindowTextW(m_verdict, result.verdict.c_str());
    SetWindowTextW(m_advice, result.advice.c_str());
    InvalidateRect(m_verdict, nullptr, TRUE);
}

void MainWindow::onValidationFailed(const std::wstring &message)
{
    MessageBoxW(m_hwnd, message.c_str(), L"Некорректные данные", MB_OK |
MB_ICONWARNING);
}

void MainWindow::clearResult()
{
    m_level = PressureLevel::Unknown;
    for (HWND label : { m_age, m_idealPressure, m_current, m_deviation,
m_verdict })
        SetWindowTextW(label, ui::kEmpty);
    SetWindowTextW(m_advice, L"");
}
```

```
// ------------------------------------------------------------------------
-
// Оконная процедура
// ------------------------------------------------------------------------
-

LRESULT CALLBACK MainWindow::windowProc(HWND hwnd, UINT message, WPARAM
wParam, LPARAM lParam)
{
    MainWindow *self = nullptr;
    if (message == WM_NCCREATE) {
        auto *cs = reinterpret_cast<CREATESTRUCTW *>(lParam);
        self = static_cast<MainWindow *>(cs->lpCreateParams);
        self->m_hwnd = hwnd;
        SetWindowLongPtrW(hwnd, GWLP_USERDATA,
reinterpret_cast<LONG_PTR>(self));
    } else {
        self = reinterpret_cast<MainWindow *>(GetWindowLongPtrW(hwnd,
GWLP_USERDATA));
    }

    if (self)
        return self->handleMessage(message, wParam, lParam);
    return DefWindowProcW(hwnd, message, wParam, lParam);
}

LRESULT MainWindow::handleMessage(UINT message, WPARAM wParam, LPARAM lParam)
{
    switch (message) {
    case WM_CREATE:
        createControls();
        return 0;

    case WM_COMMAND:
        if (LOWORD(wParam) == kEnterButtonId && HIWORD(wParam) == BN_CLICKED)
{
            if (onEnterDataRequested)
                onEnterDataRequested();
            return 0;
        }
        break;

    case WM_CTLCOLORSTATIC: {
        // Белый фон для меток; цвет оценки зависит от уровня давления.
        HDC dc = reinterpret_cast<HDC>(wParam);
        SetBkMode(dc, TRANSPARENT);
        if (reinterpret_cast<HWND>(lParam) == m_verdict) {
            switch (m_level) {
            case PressureLevel::High:   SetTextColor(dc, RGB(196, 30, 30));
break;
            case PressureLevel::Low:    SetTextColor(dc, RGB(25, 90, 200));
break;
            case PressureLevel::Normal: SetTextColor(dc, RGB(20, 135, 60));
break;
            default: break;
            }
        }
        return reinterpret_cast<LRESULT>(GetSysColorBrush(COLOR_WINDOW));
    }

    case WM_DESTROY:
```

```
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProcW(m_hwnd, message, wParam, lParam);
}
```

Файл src/view/inputdialog.h

```
#ifndef INPUTDIALOG_H
#define INPUTDIALOG_H

#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>

#include "model/pressuremodel.h"

// Отдельное модальное окно ввода данных.
// Ничего не вычисляет и не проверяет — только собирает введённый текст.
class InputDialog
{
public:
    explicit InputDialog(HWND owner);

    InputDialog(const InputDialog &) = delete;
    InputDialog &operator=(const InputDialog &) = delete;

    void setInput(const PressureInput &input); // восстановление прошлого
ввода
    PressureInput input() const { return m_input; } // выдача введённого
контроллеру

    // Показывает окно модально. true — пользователь нажал «Рассчитать».
    bool exec();

private:
    static LRESULT CALLBACK windowProc(HWND hwnd, UINT message, WPARAM
wParam, LPARAM lParam);
    LRESULT handleMessage(UINT message, WPARAM wParam, LPARAM lParam);

    void createControls();
    void readInput();

    HWND m_owner;
    HWND m_hwnd = nullptr;
    HFONT m_font = nullptr;

    // Дата рождения вводится тремя отдельными полями:
    // стандартный элемент ввода даты по условию использовать нельзя.
    HWND m_day       = nullptr;
    HWND m_month     = nullptr;
    HWND m_year      = nullptr;
    HWND m_weight    = nullptr;
    HWND m_systolic  = nullptr;
    HWND m_diastolic = nullptr;

    PressureInput m_input;
    bool m_accepted = false;
```

```
    bool m_finished = false;
};

#endif // INPUTDIALOG_H
```

Файл src/view/inputdialog.cpp

```
#include "view/inputdialog.h"

#include <commctrl.h>

#include "view/uihelpers.h"

namespace {

const wchar_t kClassName[] = L"PressureCalculatorInputDialog";

HWND createEdit(HWND parent, int x, int y, int w, int maxLength, const
wchar_t *cue)
{
    HWND edit = ui::createChild(parent, L"EDIT", L"",
                                WS_TABSTOP | ES_AUTOHSCROLL, x, y, w, 23, 0,
WS_EX_CLIENTEDGE);
    SendMessageW(edit, EM_SETLIMITTEXT, static_cast<WPARAM>(maxLength), 0);
    if (cue)
        SendMessageW(edit, EM_SETCUEBANNER, FALSE,
reinterpret_cast<LPARAM>(cue));
    return edit;
}

} // namespace

InputDialog::InputDialog(HWND owner)
    : m_owner(owner)
{
}

void InputDialog::setInput(const PressureInput &input)
{
    m_input = input;
}

bool InputDialog::exec()
{
    static bool registered = false;
    if (!registered) {
        WNDCLASSEXW wc{};
        wc.cbSize        = sizeof(wc);
        wc.lpfnWndProc   = &InputDialog::windowProc;
        wc.hInstance     = GetModuleHandleW(nullptr);
        wc.hCursor       = LoadCursorW(nullptr, IDC_ARROW);
        wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_WINDOW + 1);
        wc.lpszClassName = kClassName;
        RegisterClassExW(&wc);
        registered = true;
    }

    const DWORD style   = WS_POPUP | WS_CAPTION | WS_SYSMENU;
    const DWORD exStyle = WS_EX_DLGMODALFRAME | WS_EX_CONTROLPARENT;
```

```
    RECT rect{ 0, 0, ui::scale(426), ui::scale(220) };
    AdjustWindowRectEx(&rect, style, FALSE, exStyle);
    const int width  = rect.right - rect.left;
    const int height = rect.bottom - rect.top;

    // Окно размещается по центру главного окна.
    RECT ownerRect{};
    GetWindowRect(m_owner, &ownerRect);
    const int x = ownerRect.left + (ownerRect.right - ownerRect.left - width)
/ 2;
    const int y = ownerRect.top + (ownerRect.bottom - ownerRect.top - height)
/ 2;

    m_accepted = false;
    m_finished = false;
    m_hwnd = CreateWindowExW(exStyle, kClassName, L"Ввод данных", style,
                             x, y, width, height, m_owner, nullptr,
                             GetModuleHandleW(nullptr), this);
    if (!m_hwnd)
        return false;

    // Модальность: главное окно блокируется, пока открыт диалог.
    EnableWindow(m_owner, FALSE);
    ShowWindow(m_hwnd, SW_SHOW);
    SetFocus(m_day);
    SendMessageW(m_day, EM_SETSEL, 0, -1);

    MSG msg{};
    while (!m_finished) {
        const BOOL got = GetMessageW(&msg, nullptr, 0, 0);
        if (got == 0) {                         // пришёл WM_QUIT — вернуть его
в очередь
            PostQuitMessage(static_cast<int>(msg.wParam));
            break;
        }
        if (got == -1)
            break;
        if (!IsDialogMessageW(m_hwnd, &msg)) { // Tab, Enter, Esc как в
обычном диалоге
            TranslateMessage(&msg);
            DispatchMessageW(&msg);
        }
    }

    EnableWindow(m_owner, TRUE);
    SetActiveWindow(m_owner);
    DestroyWindow(m_hwnd);
    m_hwnd = nullptr;
    if (m_font) {
        DeleteObject(m_font);
        m_font = nullptr;
    }
    return m_accepted;
}

void InputDialog::createControls()
{
    m_font = ui::createUiFont();

    ui::createChild(m_hwnd, L"STATIC", L"Дата рождения (дд.мм.гггг):", 0, 16,
19, 228, 20);
```

```
    m_day   = createEdit(m_hwnd, 250, 16, 40, 2, L"дд");
    ui::createChild(m_hwnd, L"STATIC", L".", SS_CENTER, 290, 19, 12, 20);
    m_month = createEdit(m_hwnd, 302, 16, 40, 2, L"мм");
    ui::createChild(m_hwnd, L"STATIC", L".", SS_CENTER, 342, 19, 12, 20);
    m_year  = createEdit(m_hwnd, 354, 16, 56, 4, L"гггг");

    ui::createChild(m_hwnd, L"STATIC", L"Вес, кг:", 0, 16, 51, 228, 20);
    m_weight = createEdit(m_hwnd, 250, 48, 160, 6, L"например, 75");

    ui::createChild(m_hwnd, L"STATIC", L"Верхнее (систолическое) давление:",
0, 16, 83, 228, 20);
    m_systolic = createEdit(m_hwnd, 250, 80, 160, 3, L"например, 120");

    ui::createChild(m_hwnd, L"STATIC", L"Нижнее (диастолическое) давление:",
0, 16, 115, 228, 20);
    m_diastolic = createEdit(m_hwnd, 250, 112, 160, 3, L"например, 80");

    ui::createChild(m_hwnd, L"STATIC", L"Давление указывается в мм рт. ст.",
0, 16, 148, 394, 20);

    ui::createChild(m_hwnd, L"BUTTON", L"Рассчитать",
                    WS_TABSTOP | BS_DEFPUSHBUTTON, 226, 178, 90, 28, IDOK);
    ui::createChild(m_hwnd, L"BUTTON", L"Отмена",
                    WS_TABSTOP | BS_PUSHBUTTON, 322, 178, 88, 28, IDCANCEL);

    ui::applyFontToChildren(m_hwnd, m_font);

    // Восстановление последних введённых данных.
    SetWindowTextW(m_day,       m_input.day.c_str());
    SetWindowTextW(m_month,     m_input.month.c_str());
    SetWindowTextW(m_year,      m_input.year.c_str());
    SetWindowTextW(m_weight,    m_input.weight.c_str());
    SetWindowTextW(m_systolic,  m_input.systolic.c_str());
    SetWindowTextW(m_diastolic, m_input.diastolic.c_str());
}

void InputDialog::readInput()
{
    m_input.day       = ui::windowText(m_day);
    m_input.month     = ui::windowText(m_month);
    m_input.year      = ui::windowText(m_year);
    m_input.weight    = ui::windowText(m_weight);
    m_input.systolic  = ui::windowText(m_systolic);
    m_input.diastolic = ui::windowText(m_diastolic);
}

LRESULT CALLBACK InputDialog::windowProc(HWND hwnd, UINT message, WPARAM
wParam, LPARAM lParam)
{
    InputDialog *self = nullptr;
    if (message == WM_NCCREATE) {
        auto *cs = reinterpret_cast<CREATESTRUCTW *>(lParam);
        self = static_cast<InputDialog *>(cs->lpCreateParams);
        self->m_hwnd = hwnd;
        SetWindowLongPtrW(hwnd, GWLP_USERDATA,
reinterpret_cast<LONG_PTR>(self));
    } else {
        self = reinterpret_cast<InputDialog *>(GetWindowLongPtrW(hwnd,
GWLP_USERDATA));
    }
```

```
    if (self && self->m_hwnd == hwnd)
        return self->handleMessage(message, wParam, lParam);
    return DefWindowProcW(hwnd, message, wParam, lParam);
}

LRESULT InputDialog::handleMessage(UINT message, WPARAM wParam, LPARAM
lParam)
{
    switch (message) {
    case WM_CREATE:
        createControls();
        return 0;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDOK:          // «Рассчитать» или Enter
            readInput();
            m_accepted = true;
            m_finished = true;
            return 0;
        case IDCANCEL:      // «Отмена» или Esc
            m_finished = true;
            return 0;
        }
        break;

    case WM_CLOSE:          // крестик в заголовке
        m_finished = true;
        return 0;

    case WM_CTLCOLORSTATIC: {
        HDC dc = reinterpret_cast<HDC>(wParam);
        SetBkMode(dc, TRANSPARENT);
        return reinterpret_cast<LRESULT>(GetSysColorBrush(COLOR_WINDOW));
    }
    }
    return DefWindowProcW(m_hwnd, message, wParam, lParam);
}
```

Файл src/controller/pressurecontroller.h

```
#ifndef PRESSURECONTROLLER_H
#define PRESSURECONTROLLER_H

#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>

class PressureModel;

// Контроллер: принимает намерения пользователя, показывает окно ввода
// и передаёт собранные данные в модель. Результат не отображает —
// это делает представление по уведомлениям активной модели.
class PressureController
{
public:
    PressureController(PressureModel *model, HWND dialogOwner);
```

```
    void onEnterDataRequested();

private:
    PressureModel *m_model;
    HWND           m_dialogOwner;
};

#endif // PRESSURECONTROLLER_H
```

Файл src/controller/pressurecontroller.cpp

```
#include "controller/pressurecontroller.h"

#include "model/pressuremodel.h"
#include "view/inputdialog.h"

PressureController::PressureController(PressureModel *model, HWND
dialogOwner)
    : m_model(model)
    , m_dialogOwner(dialogOwner)
{
}

void PressureController::onEnterDataRequested()
{
    InputDialog dialog(m_dialogOwner);

    // Восстановление последних введённых данных из модели.
    dialog.setInput(m_model->input());

    if (!dialog.exec())
        return; // пользователь нажал «Отмена»

    m_model->submit(dialog.input());
}
```

Файл src/main.cpp

```
#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>
#include <commctrl.h>

#include "controller/pressurecontroller.h"
#include "model/pressuremodel.h"
#include "view/mainwindow.h"

#ifdef _MSC_VER
#pragma comment(lib, "comctl32.lib")
#endif

int WINAPI wWinMain(HINSTANCE instance, HINSTANCE, PWSTR, int showCommand)
{
    // Подключение современных элементов управления Windows (Common Controls
6).
    INITCOMMONCONTROLSEX icc{ sizeof(icc), ICC_STANDARD_CLASSES };
    InitCommonControlsEx(&icc);
```

```
    PressureModel model;
    MainWindow window(&model);
    if (!window.create(instance, showCommand))
        return 1;

    PressureController controller(&model, window.hwnd());

    // Единственная связь View -> Controller: намерение пользователя.
    window.onEnterDataRequested = [&controller] {
controller.onEnterDataRequested(); };

    // Цикл обработки сообщений.
    MSG msg{};
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        if (!IsDialogMessageW(window.hwnd(), &msg)) {
            TranslateMessage(&msg);
            DispatchMessageW(&msg);
        }
    }
    return static_cast<int>(msg.wParam);
}
```